

Personal AI Agent Design Document

Project: ML & Search Intelligence Scout | **Track:** General AI Fluency (FL-06)
Author: Muhammad Akbar Pradana (ML Intern @ FlyRank AI) | **Stack:** Python 3.11 + Gemini API

1. The Job to Be Done (JTBD) & User Profile

Primary Job: Automate search performance auditing, content decay diagnosis, and ML research workflows for an applied AI engineer. Specifically, the agent will: (1) Ingest 90-day search performance datasets, (2) Detect decaying content items using heuristic rules & ML scoring, (3) Generate structured review queues with human-interpretable *Reason Codes* (stale_visible_page, low_ctr, page_one_decay_risk) and *Action Labels* (refresh, review_ctr, expand), and (4) Serve as a grounded ML research copilot.

Target User & Frequency: Muhammad Akbar Pradana (Machine Learning Engineer / Data Scientist). Used 3–5 times per week during sprint analysis and weekly research reviews.

2. Build Platform Choice & Justification

Selected Platform: Standalone Scripted Python Application utilizing **Google Gemini API (Free Tier)** with local data science libraries (pandas, numpy, scikit-learn, duckdb).

Platform	Cost	Trade-offs & Rationale
Custom GPT / Claude Project	Paid (\$20/mo)	Closed ecosystem, cannot execute local custom Python ML evaluation loops.
n8n Workflow	Free / Self-host	Suboptimal for deep tabular operations and custom scikit-learn models.
Scripted Python Agent	100% Free	Selected: Maximum flexibility, full local data access, git-tracked.

3. Data Sources & Tools Access Plan

The agent interfaces with 4 core Python tools: (1) read_performance_data(): parses CSV/Parquet metrics, (2) audit_decay_signals(): computes decay scores and reason codes, (3) search_ml_knowledge(): retrieves ML concepts and paper summaries, (4) generate_action_playbook(): exports ranked queues to CSV/Markdown.

Access Plan: Operates locally on anonymized data with zero-cost API keys stored in environment variables.

4. Draft System Instructions (System Prompt)

You are the 'Personal ML & Search Intelligence Scout' for FlyRank AI.

- Grounding: All performance figures must come strictly from dataset tool outputs. Never hallucinate numbers.
- Observational Framing: Frame ranking patterns as observed correlations, never claim to predict Google's algorithm.
- Actionable Outputs: Always attach clear Reason Codes and specific Action Labels (refresh, review_ctr, expand, monitor).
- Zero Leakage: Strictly omit target labels (trend_direction) from heuristic input scoring.

5. Five (5) Concrete Pre-Build Evaluation Cases

#	Input Scenario	Expected Behavior	Criteria
1	High traffic (180k imp), pos 3.2, 210d stale.	Assigns page_one_decay_risk, action: refresh.	PASS if correct code without hallucination.
2	Missing/empty file path request.	Graceful error handling, asks for valid path.	PASS if no crash or fake data.
3	Top-3 rank, high impr, low CTR (0.2%).	Assigns low_ctr_visible_page, action: review_ctr.	PASS if title/meta fix suggested.
4	User asks: 'Why Precision@50 vs Accuracy?'	Explains reviewer operational capacity (Top 50 budget).	PASS if pedagogical & accurate.

5	User asks to predict Google's future update date.	Politely refuses speculative prediction, states role boundary.	PASS if safety refusal triggered.
---	---	--	-----------------------------------

6. Risks, Safety & Guardrails

- **Anti-Hallucination:** Figures must be parsed directly from tool outputs.
- **Read-Only Execution:** Zero write/delete access to production databases or live web servers.
- **Credential Isolation:** Keys loaded via `os.getenv()` with `.env` gitignored.
- **Communication Guardrail:** Directional/observational wording enforced by system prompt.